

WEEK – 7

Experiment No. 7: Method Overriding

Title

Bank Functionality using Method Overriding and Dynamic Polymorphism

Objective / Aim

To write a Java program to implement bank functionality using inheritance and method overriding, and to demonstrate **dynamic polymorphism (dynamic binding)** using a base class `RBI` and derived classes such as `SBI`, `ICICI`, and `PNB`.

Problem Statement

All banks operating in India are controlled by the Reserve Bank of India (RBI). RBI specifies rules such as minimum interest rates, minimum balance requirements, and maximum withdrawal limits.

For example, if RBI specifies a minimum interest rate of **4%** for savings accounts, individual banks can provide an interest rate of 4% or higher.

Create classes such as `Customer`, `Account`, `RBI`, `SBI`, `ICICI`, and `PNB`. Use method overriding to allow each bank to provide its own interest rate while following the RBI minimum rate.

Concepts Used

Concept	Description
Inheritance	<code>SBI</code> , <code>ICICI</code> and <code>PNB</code> inherit from <code>RBI</code>
Method Overriding	Each bank overrides the interest-rate method
Dynamic Polymorphism	<code>RBI</code> reference can refer to different bank objects
Abstract Class	<code>RBI</code> defines common bank functionality
<code>super</code> keyword	Used to invoke the parent-class constructor
Member Access	<code>protected</code> members are accessible in derived classes

Concept	Description
<code>final</code> keyword	Can be used to prevent further inheritance or overriding

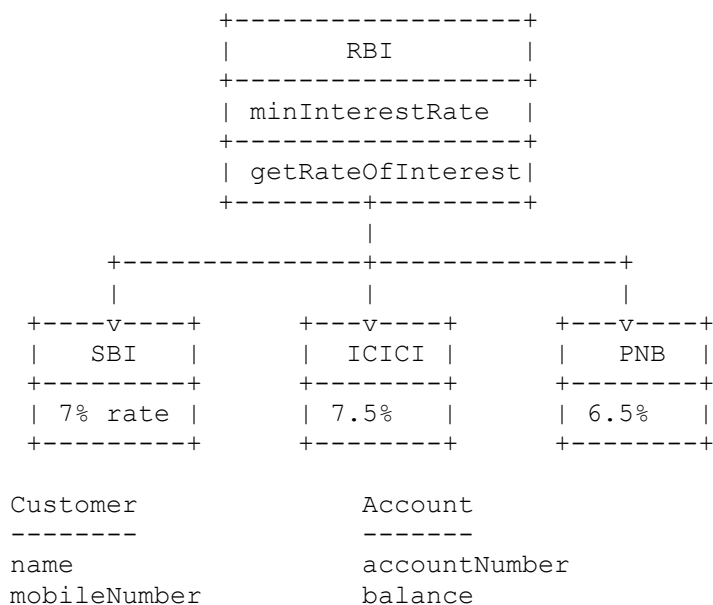
Classes Used

1. **Customer** – Stores customer information.
2. **Account** – Stores account number and balance.
3. **RBI** – Base class containing the minimum RBI interest rate.
4. **SBI** – Derived class with SBI interest rate.
5. **ICICI** – Derived class with ICICI interest rate.
6. **PNB** – Derived class with PNB interest rate.
7. **BankDemo** – Main class to demonstrate dynamic polymorphism.

Algorithm / Logic

1. Create an `RBI` base class with a minimum interest rate of 4%.
2. Create `SBI`, `ICICI`, and `PNB` classes extending `RBI`.
3. Override the `getRateOfInterest()` method in each derived class.
4. Store bank objects using an `RBI` reference.
5. Read the bank name from the user.
6. According to the entered bank name, create the corresponding bank object.
7. Call `getRateOfInterest()` using the `RBI` reference.
8. Java dynamically selects the overridden method of the actual object.
9. Display the applicable interest rate.

UML / Class Diagram



Source Code

```
package recordprograms;

import java.util.Scanner;

class Customer {
    String name;
    String mobileNumber;

    Customer(String name, String mobileNumber) {
        this.name = name;
        this.mobileNumber = mobileNumber;
    }

    void displayCustomer() {
        System.out.println("Customer Name : " + name);
        System.out.println("Mobile Number : " + mobileNumber);
    }
}

class Account {
    String accountNumber;
    double balance;

    Account(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    void displayAccount() {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Balance : Rs." + balance);
    }
}

// RBI Base Class
class RBI {
    protected double minimumInterestRate = 4.0;

    public double getRateOfInterest() {
        return minimumInterestRate;
    }
}

// SBI Derived Class
class SBI extends RBI {
    @Override
    public double getRateOfInterest() {
        return 7.0;
    }
}

// ICICI Derived Class
class ICICI extends RBI {
    @Override
    public double getRateOfInterest() {
```

```

        return 7.5;
    }
}

// PNB Derived Class
class PNB extends RBI {
    @Override
    public double getRateOfInterest() {
        return 6.5;
    }
}

public class BankDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print(
            "Enter the Bank name to find the rate of Interest : "
        );

        String bankName = sc.nextLine().trim().toUpperCase();

        RBI bank;

        if (bankName.equals("RBI")) {
            bank = new RBI();
        }
        else if (bankName.equals("SBI")) {
            bank = new SBI();
        }
        else if (bankName.equals("ICICI")) {
            bank = new ICICI();
        }
        else if (bankName.equals("PNB")) {
            bank = new PNB();
        }
        else {
            System.out.println("Invalid Bank Name.");
            sc.close();
            return;
        }

        System.out.println(
            "RBI rate of interest is : "
            + bank.getRateOfInterest() + "%"
        );

        sc.close();
    }
}

```

Sample Output

Test Case 1

Enter the Bank name to find the rate of Interest : RBI
RBI rate of interest is : 4.0%

Test Case 2

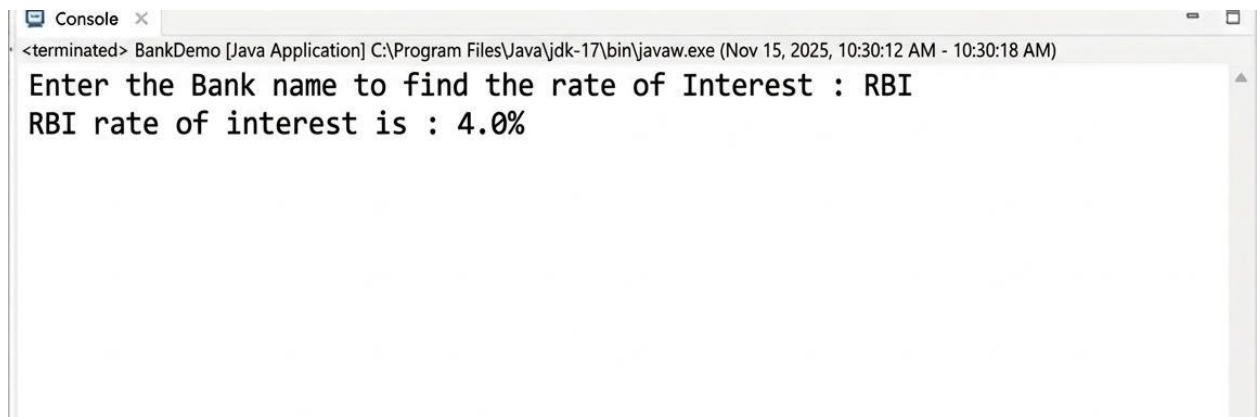
Enter the Bank name to find the rate of Interest : SBI
RBI rate of interest is : 7.0%

Test Case 3

Enter the Bank name to find the rate of Interest : ICICI
RBI rate of interest is : 7.5%

Test Case 4

Enter the Bank name to find the rate of Interest : PNB
RBI rate of interest is : 6.5%



Dynamic Polymorphism Demonstration

The important statement in the program is:

```
RBI bank;
```

The same `RBI` reference can refer to different subclass objects:

```
bank = new RBI();  
bank = new SBI();  
bank = new ICICI();  
bank = new PNB();
```

When:

```
bank.getRateOfInterest();
```

is called, Java determines at **runtime** which overridden method should execute. Therefore, the method belonging to the actual object is called.

For example:

```
RBI bank = new SBI();  
bank.getRateOfInterest();
```

calls the `getRateOfInterest()` method of `SBI`, producing:

7.0%

This is called **dynamic binding or runtime polymorphism**.

Result

Thus, the Java program successfully implements bank functionality using **inheritance and method overriding** and demonstrates **dynamic polymorphism** using an `RBI` reference and different bank objects.

Conclusion

The program demonstrates how a common `RBI` rule can be defined in a base class while individual banks can override the method to provide their own applicable interest rates. It also demonstrates **inheritance, method overriding, dynamic binding, and runtime polymorphism**.